



Linux
Professional
Institute

101.2 Lektion 1

Zertifikat:	LPIC-1
Version:	5.0
Thema:	101 Systemarchitektur
Lernziel:	101.2 Das System starten
Lektion:	1 von 1

Einführung

Um die Maschine zu steuern, muss die Hauptkomponente des Betriebssystems — der Kernel — von einem Programm namens *Bootloader* geladen werden, das seinerseits von einer vorinstallierten Firmware wie BIOS oder UEFI geladen wird. Der Bootloader kann so angepasst werden, dass er Parameter an den Kernel übergibt, z.B. welche Partition das Root-Dateisystem enthält oder in welchem Modus das Betriebssystem ausgeführt werden soll. Nach dem Laden setzt der Kernel den Bootprozess fort, indem er die Hardware identifiziert und konfiguriert. Anschließend ruft der Kernel das Dienstprogramm auf, welches für den Start und die Verwaltung der Systemdienste verantwortlich ist.

NOTE

Auf einigen Linux-Distributionen können Befehle, die in dieser Lektion ausgeführt werden, Root-Rechte erfordern.

BIOS oder UEFI

Die Prozeduren, die von x86-Maschinen ausgeführt werden, um den Bootloader zu starten, unterscheiden sich je nachdem ob BIOS oder UEFI verwendet wird. Das BIOS, kurz für *Basic Input/Output System*, ist ein Programm, das in einem nichtflüchtigen Speicherchip auf der Hauptplatine gespeichert ist und jedes Mal ausgeführt wird, wenn der Computer eingeschaltet wird. Diese Art von Programm wird *Firmware* genannt und sein Speicherort ist von den anderen Speichergeräten, die das System möglicherweise hat, getrennt. Das BIOS geht davon aus, dass die ersten 440 Bytes des ersten Speichergeräts — entsprechend der im BIOS-Konfigurationsprogramm festgelegten Reihenfolge — die erste Stufe des Bootloaders (auch *bootstrap* genannt) sind. Die ersten 512 Bytes eines Speichergeräts werden bei Speichergeräten mit dem Standard-DOS-Partitionsschema als MBR (*Master Boot Record*) bezeichnet und enthalten neben der ersten Stufe des Bootloaders auch die Partitionstabelle. Wenn der MBR nicht die richtigen Daten enthält, kann das System nicht booten, es sei denn, es wird eine alternative Methode verwendet.

Im Allgemeinen sind die Vorbereitungsschritte zum Booten eines mit BIOS ausgestatteten Systems folgende:

1. Der POST-Prozess (*Power-on Self-Test*) wird ausgeführt, um einfache Hardwarefehler zu identifizieren, sobald das Gerät eingeschaltet wird.
2. Das BIOS aktiviert die grundlegenden Komponenten zum Laden des Systems, wie Videoausgabe, Tastatur und Speichermedien.
3. Das BIOS lädt die erste Stufe des Bootloaders aus dem MBR (die ersten 440 Bytes des ersten Geräts, wie im BIOS-Konfigurationsprogramm definiert).
4. Die erste Stufe des Bootloaders ruft die zweite Stufe des Bootloaders auf, die für die Präsentation von Bootoptionen und das Laden des Kernels verantwortlich ist.

UEFI, kurz für *Unified Extensible Firmware Interface*, unterscheidet sich in einigen wesentlichen Punkten vom BIOS. Wie das BIOS ist auch das UEFI eine Firmware, aber es kann Partitionen identifizieren und eine Vielzahl von Dateisystemen lesen, die sich darin befinden. UEFI ist nicht auf den MBR angewiesen und berücksichtigt nur die Einstellungen, die in ihrem nichtflüchtigen Speicher (*NVRAM*) gespeichert sind, der an die Hauptplatine angeschlossen ist. Diese Definitionen geben den Speicherort der UEFI-kompatiblen Programme, *EFI-Anwendungen* genannt, an, die automatisch ausgeführt oder über ein Bootmenü aufgerufen werden. EFI-Anwendungen können Bootloader, Betriebssystemselektoren, Werkzeuge für Systemdiagnose und -reparatur usw. sein. Sie müssen sich in einer konventionellen Speichergerätepartition und in einem kompatiblen Dateisystem befinden. Die standardmäßigen kompatiblen Dateisysteme sind FAT12, FAT16 und FAT32 für Blockgeräte und ISO-9660 für optische Medien. Dieser Ansatz ermöglicht die Implementierung viel ausgefeilterer Werkzeuge, welche mit einem BIOS möglich sind.

Die Partition, welche die EFI-Anwendungen enthält, wird *EFI-System-Partition* oder einfach ESP genannt. Diese Partition darf nicht mit anderen Systemdateisystemen, wie dem Root-Dateisystem oder Benutzerdatendateisystemen, geteilt werden. Das EFI-Verzeichnis in der ESP-Partition enthält die Anwendungen, auf welche die im NVRAM gespeicherten Einträge verweisen.

Im Allgemeinen sind die vor dem Betriebssystemstart durchzuführenden Schritte auf einem UEFI-System:

1. Der POST-Prozess (*Power-on Self-Test*) wird ausgeführt, um einfache Hardwarefehler zu identifizieren, sobald das Gerät eingeschaltet wird.
2. Das UEFI aktiviert die grundlegenden Komponenten zum Laden des Systems, wie Videoausgabe, Tastatur und Speichermedien.
3. Die UEFI-Firmware liest die im NVRAM gespeicherten Definitionen, um die vordefinierte EFI-Anwendung auszuführen, die im Dateisystem der ESP-Partition gespeichert ist. Normalerweise ist die vordefinierte EFI-Anwendung ein Bootloader.
4. Wenn die vordefinierte EFI-Anwendung ein Bootloader ist, lädt dieser anschließend den Kernel, um das Betriebssystem zu starten.

Der UEFI-Standard unterstützt auch eine Funktion namens *Secure Boot*, die nur die Ausführung von signierten EFI-Anwendungen erlaubt, d.h. von EFI-Anwendungen, die vom Hardwarehersteller autorisiert sind. Diese Funktion erhöht den Schutz vor bösartiger Software, kann aber die Installation von Betriebssystemen erschweren, welche nicht durch die Herstellergarantie abgedeckt sind.

Der Bootloader

Der beliebteste Bootloader für Linux in der x86-Architektur ist GRUB (*Grand Unified Bootloader*). Sobald er vom BIOS oder vom UEFI aufgerufen wird, zeigt GRUB eine Liste der zum Booten verfügbaren Betriebssysteme an. Manchmal erscheint die Liste nicht automatisch, aber sie kann durch Drücken von `Shift` aufgerufen werden, während GRUB vom BIOS aufgerufen wird. In UEFI-Systemen sollte stattdessen die Taste `Esc` verwendet werden.

Im GRUB-Menü ist es möglich zu wählen, welcher der installierten Kernel geladen werden soll und welche Parameter diesem übergeben werden. Die meisten Kernelparameter folgen dem Muster `option=value`. Einige der nützlichsten Kernelparameter sind:

acpi

Aktiviert/deaktiviert die ACPI-Unterstützung. `acpi=off` deaktiviert die Unterstützung für ACPI.

init

Legt einen alternativen Systeminitiator fest. Zum Beispiel wird `init=/bin/bash` die Shell Bash als Initiator festlegen. Das bedeutet, dass eine Shell-Sitzung direkt nach dem Kernel-Bootprozess gestartet wird.

systemd.unit

Legt das zu aktivierende *systemd* Ziel fest. Zum Beispiel `systemd.unit=graphical.target`. Systemd akzeptiert auch die numerischen Runlevels, wie sie für SysV definiert sind. Um z.B. den Runlevel 1 zu aktivieren, ist es nur erforderlich, die Zahl 1 oder den Buchstaben S (kurz für “single”) als Kernelparameter hinzuzufügen.

mem

Legt die Menge des verfügbaren RAM für das System fest. Dieser Parameter ist für virtuelle Maschinen nützlich, um zu begrenzen, wie viel RAM jedem Gast zur Verfügung steht. Die Verwendung von `mem=512M` begrenzt die Menge des verfügbaren RAM für ein bestimmtes Gastsystem auf 512 Megabyte.

maxcpus

Begrenzt die Anzahl der für das System sichtbaren Prozessoren (oder Prozessorkerne) in symmetrischen Mehrprozessormaschinen. Dies ist auch für virtuelle Maschinen nützlich. Ein Wert von 0 schaltet die Unterstützung für Mehrkernprozessor-Maschinen ab und hat die gleiche Wirkung wie der Kernelparameter `nosmp`. Der Parameter `maxcpus=2` begrenzt die Anzahl der dem Betriebssystem zur Verfügung stehenden Prozessoren auf zwei.

quiet

Blendet die meisten Bootmeldungen aus.

vga

Wählt einen Videomodus aus. Der Parameter `vga=ask` zeigt eine Liste der verfügbaren Modi zur Auswahl an.

root

Legt die Root-Partition fest, die sich von der im Bootloader vorkonfigurierten Partition unterscheidet. Zum Beispiel `root=/dev/sda3`.

rootflags

Einhängeoptionen für das Root-Dateisystem.

ro

Sorgt für die ausschließliche Lesbarkeit (read only) des Root-Dateisystems.

rw

Erlaubt das Schreiben in das Root-Dateisystem während des erstmaligen Einhängens.

Eine Änderung der Kernelparameter ist normalerweise nicht erforderlich, kann aber nützlich sein, um betriebssystembezogene Probleme zu erkennen und zu lösen. Kernelparameter müssen der Datei `/etc/default/grub` in der Zeile `GRUB_CMDLINE_LINUX` hinzugefügt werden, um sie über Neustarts hinweg persistent zu machen. Eine neue Konfigurationsdatei für den Bootloader muss bei jeder Änderung von `/etc/default/grub` erzeugt werden, was durch den Befehl `grub-mkconfig -o /boot/grub/grub.cfg` erreicht wird. Sobald das Betriebssystem läuft, stehen die Kernelparameter, die zum Laden der aktuellen Sitzung verwendet werden, in der Datei `/proc/cmdline` zum Einlesen zur Verfügung.

NOTE Die Konfiguration von GRUB wird in einer späteren Lektion behandelt.

Systeminitialisierung

Abgesehen vom Kernel hängt das Betriebssystem von weiteren Komponenten ab. Viele dieser Komponenten werden während des Systeminitialisierungsprozesses geladen und reichen von einfachen Shell-Skripten bis hin zu komplexeren Dienstprogrammen. Skripte werden oft verwendet, um kurzlebige Aufgaben auszuführen, die während des Systeminitialisierungsprozesses ausgeführt und beendet werden. Dienste, auch als *Daemons* bekannt, können die ganze Zeit aktiv sein, da sie für intrinsische Aspekte des Betriebssystems verantwortlich sein können.

Die Vielfalt der Möglichkeiten, Startskripte und Daemons mit den unterschiedlichsten Eigenschaften in eine Linux-Distribution einzubauen, ist riesig, eine Tatsache, die in der Vergangenheit die Entwicklung einer einzigen Lösung behindert hat, die die Erwartungen der Maintainer und Benutzer aller Linux-Distributionen erfüllt. Allerdings wird jedes Werkzeug, das die Maintainer einer Distribution für diese Funktion ausgewählt haben, zumindest in der Lage sein, Systemdienste zu starten, zu stoppen und neu zu starten. Diese Aktionen werden oft vom System selbst ausgeführt, z. B. nach einem Softwareupdate, aber der Systemadministrator wird den Dienst fast immer manuell neu starten müssen, nachdem er Änderungen an seiner Konfigurationsdatei vorgenommen hat.

Für einen Systemadministrator ist es auch praktisch, nach den jeweiligen Anforderungen einen bestimmten Satz von Daemons aktivieren zu können. Es sollte z.B. möglich sein, nur ein Minimum an Diensten laufen zu lassen, um Systemwartungsaufgaben durchzuführen.

NOTE Streng genommen sind das Betriebssystem nur der Kernel und seine Komponenten, die die Hardware steuern und alle Prozesse verwalten. Es ist jedoch üblich, den Begriff "Betriebssystem" weiter zu fassen, um eine ganze Gruppe

verschiedener Programmen zu bezeichnen, welche die Softwareumgebung bilden, in welcher der Benutzer die grundlegenden Aufgaben ausführen kann.

Die Initialisierung des Betriebssystems beginnt, wenn der Bootloader den Kernel in den RAM-Speicher lädt. Dann übernimmt der Kernel die Kontrolle über die CPU und beginnt mit der Erkennung und Einrichtung der grundlegenden Aspekte des Betriebssystems, wie grundlegende Hardwarekonfiguration und Speicheradressierung.

Der Kernel öffnet dann das *initramfs* (*initiales RAM-Dateisystem*). Das initiale RAM Dateisystem ist ein Archiv, das ein Dateisystem enthält, das während des Bootvorgangs als temporäres Root-Dateisystem verwendet wird. Der Hauptzweck der Datei *initramfs* besteht darin, die erforderlichen Module bereitzustellen, damit der Kernel auf das “echte” Root-Dateisystem des Betriebssystems zugreifen kann.

Sobald das Root-Dateisystem verfügbar ist, wird der Kernel alle in */etc/fstab* konfigurierten Dateisysteme einbinden und dann das erste Programm, ein Dienstprogramm namens *init*, ausführen. Das Programm *init* ist für die Ausführung aller Initialisierungsskripte und System-Daemons verantwortlich. Es gibt verschiedene Implementierungen solcher Systeminitiatoren neben dem traditionellen *init*, wie beispielsweise *systemd* und *Upstart*. Sobald das *init*-Programm geladen ist, wird das *initramfs* aus dem RAM entfernt.

SysV Standard

Ein Servicemanager, der auf dem SysVinit-Standard basiert, steuert, welche Daemons und Ressourcen verfügbar sein werden, indem er das Konzept der *Runlevel* einsetzt. Die Runlevel sind von 0 bis 6 nummeriert und werden von den Distributionsbetreuern für bestimmte Zwecke entworfen. Die einzigen Runlevel-Definitionen, die von allen Distributionen gemeinsam genutzt werden, sind die Runlevels 0, 1 und 6.

systemd

Systemd ist ein moderner System- und Dienstmanager mit einer Kompatibilitätsschicht für die Befehle von SysV und Runlevels. Systemd verwendet Sockets und D-Bus für die Dienstaktivierung, verfügt über On-Demand-Daemon-Ausführung, Prozessüberwachung mit *cgroups*, Snapshotunterstützung, Wiederherstellung von Systemsitzungen, Einhängepunktkontrolle und eine abhängigkeitsbasierte Dienstkontrolle. In den letzten Jahren haben die meisten großen Linux-Distributionen nach und nach systemd als Standardsystemmanager übernommen.

Upstart

Wie systemd ist Upstart ein Ersatz für *init*. Der Schwerpunkt von Upstart ist die Beschleunigung des Bootprozesses durch Parallelisierung des Ladevorgangs von Systemdiensten. Upstart wurde von Ubuntu-basierten Distributionen in früheren Versionen verwendet, ist heute aber

weitestgehend durch systemd verdrängt worden.

Initialisierungsprüfung

Während des Bootvorgangs können Fehler auftreten, die jedoch möglicherweise nicht so kritisch sind, dass diese das Betriebssystem vollständig zum Stillstand bringen. Ungeachtet dessen können diese Fehler das erwartete Verhalten des Systems beeinträchtigen. Alle Fehler führen zu Meldungen, die für zukünftige Untersuchungen verwendet werden können, da sie wertvolle Informationen darüber enthalten, wann und wie der Fehler aufgetreten ist. Selbst wenn keine Fehlermeldungen erzeugt werden, können die während des Bootvorgangs gesammelten Informationen für Abstimmungs- und Konfigurationszwecke nützlich sein.

Der Speicherbereich, in dem der Kernel seine Meldungen, einschließlich der Bootmeldungen, speichert, wird als *Kernel-Ringpuffer* bezeichnet. Die Meldungen werden im Kernel-Ringpuffer gehalten, auch wenn sie während des Initialisierungsvorgangs nicht angezeigt werden, z.B. wenn stattdessen eine Animation angezeigt wird. Der Kernel-Ringpuffer verliert jedoch alle Nachrichten, wenn das System abgeschaltet wird oder der Befehl `dmesg --clear` ausgeführt wird. Ohne Optionen zeigt der Befehl `dmesg` die aktuellen Meldungen des Kernel-Ringpuffer an:

```
$ dmesg
[ 5.262389] EXT4-fs (sda1): mounted filesystem with ordered data mode. Opts: (null)
[ 5.449712] ip_tables: (C) 2000-2006 Netfilter Core Team
[ 5.460286] systemd[1]: systemd 237 running in system mode.
[ 5.480138] systemd[1]: Detected architecture x86-64.
[ 5.481767] systemd[1]: Set hostname to <torre>.
[ 5.636607] systemd[1]: Reached target User and Group Name Lookups.
[ 5.636866] systemd[1]: Created slice System Slice.
[ 5.637000] systemd[1]: Listening on Journal Audit Socket.
[ 5.637085] systemd[1]: Listening on Journal Socket.
[ 5.637827] systemd[1]: Mounting POSIX Message Queue File System...
[ 5.638639] systemd[1]: Started Read required files in advance.
[ 5.641661] systemd[1]: Starting Load Kernel Modules...
[ 5.661672] EXT4-fs (sda1): re-mounted. Opts: errors=remount-ro
[ 5.694322] lp: driver loaded but no devices found
[ 5.702609] ppdev: user-space parallel port driver
[ 5.705384] parport_pc 00:02: reported by Plug and Play ACPI
[ 5.705468] parport0: PC-style at 0x378 (0x778), irq 7, dma 3
[PCSP,TRISTATE,COMPAT,EPP,ECP,DMA]
[ 5.800146] lp0: using parport0 (interrupt-driven).
[ 5.897421] systemd-journald[352]: Received request to flush runtime journal from PID 1
```


Die Ausgabe von `dmesg` kann Hunderte von Zeilen umfassen, daher enthält die vorherige Auflistung nur einen Auszug, der zeigt, wie der Kernel den `systemd`-Dienstmanager aufruft. Die Werte am Anfang der Zeilen sind die Anzahl der Sekunden relativ zum Beginn des Kernelladevorgangs.

In Systemen, die auf `systemd` basieren, zeigt der Befehl `journalctl` die Initialisierungsnachrichten mit den Optionen `-b`, `--boot`, `-k` oder `--dmesg` an. Der Befehl `journalctl --list-boots` zeigt eine Liste von Bootnummern relativ zum aktuellen Boot, ihren Identifikations-Hash, sowie die Zeitstempel der ersten und letzten entsprechenden Meldungen:

```
$ journalctl --list-boots
```

```
-4 9e5b3eb4952845208b841ad4dbefa1a6 Thu 2019-10-03 13:39:23 -03--Thu 2019-10-03 13:40:30 -03
-3 9e3d79955535430aa43baa17758f40fa Thu 2019-10-03 13:41:15 -03--Thu 2019-10-03 14:56:19 -03
-2 17672d8851694e6c9bb102df7355452c Thu 2019-10-03 14:56:57 -03--Thu 2019-10-03 19:27:16 -03
-1 55c0d9439bfb4e85a20a62776d0dbb4d Thu 2019-10-03 19:27:53 -03--Fri 2019-10-04 00:28:47 -03
 0 08fbbbebd9f964a74b8a02bb27b200622 Fri 2019-10-04 00:31:01 -03--Fri 2019-10-04 10:17:01 -03
```

In Systemen, die auf `systemd` basieren, werden auch vorangegangene Initialisierungsprotokolle aufbewahrt, sodass Meldungen aus früheren Betriebssystemssitzungen noch eingesehen werden können. Wenn die Optionen `-b 0` oder `--boot=0` angegeben werden, werden Meldungen für den aktuellen Bootvorgang angezeigt. Die Optionen `-b -1` oder `--boot=-1` zeigen Meldungen von der vorherigen Initialisierung an. Die Optionen `-b -2` oder `--boot=-2` zeigen die Meldungen von der Initialisierung davor an und so weiter. Der folgende Ausschnitt zeigt, wie der Kernel den `systemd`-Dienstmanager für den letzten Initialisierungsprozess aufruft:

```
$ journalctl -b 0
```

```
oct 04 00:31:01 ubuntu-host kernel: EXT4-fs (sda1): mounted filesystem with ordered data
mode. Opts: (null)
oct 04 00:31:01 ubuntu-host kernel: ip_tables: (C) 2000-2006 Netfilter Core Team
oct 04 00:31:01 ubuntu-host systemd[1]: systemd 237 running in system mode.
oct 04 00:31:01 ubuntu-host systemd[1]: Detected architecture x86-64.
oct 04 00:31:01 ubuntu-host systemd[1]: Set hostname to <torre>.
oct 04 00:31:01 ubuntu-host systemd[1]: Reached target User and Group Name Lookups.
oct 04 00:31:01 ubuntu-host systemd[1]: Created slice System Slice.
oct 04 00:31:01 ubuntu-host systemd[1]: Listening on Journal Audit Socket.
oct 04 00:31:01 ubuntu-host systemd[1]: Listening on Journal Socket.
oct 04 00:31:01 ubuntu-host systemd[1]: Mounting POSIX Message Queue File System...
oct 04 00:31:01 ubuntu-host systemd[1]: Started Read required files in advance.
oct 04 00:31:01 ubuntu-host systemd[1]: Starting Load Kernel Modules...
oct 04 00:31:01 ubuntu-host kernel: EXT4-fs (sda1): re-mounted. Opts:
commit=300,barrier=0,errors=remount-ro
```



```

oct 04 00:31:01 ubuntu-host kernel: lp: driver loaded but no devices found
oct 04 00:31:01 ubuntu-host kernel: ppdev: user-space parallel port driver
oct 04 00:31:01 ubuntu-host kernel: parport_pc 00:02: reported by Plug and Play ACPI
oct 04 00:31:01 ubuntu-host kernel: parport0: PC-style at 0x378 (0x778), irq 7, dma 3
[PCSP, TRISTATE, COMPAT, EPP, ECP, DMA]
oct 04 00:31:01 ubuntu-host kernel: lp0: using parport0 (interrupt-driven).
oct 04 00:31:01 ubuntu-host systemd-journald[352]: Journal started
oct 04 00:31:01 ubuntu-host systemd-journald[352]: Runtime journal
(/run/log/journal/abb765408f3741ae9519ab3b96063a15) is 4.9M, max 39.4M, 34.5M free.
oct 04 00:31:01 ubuntu-host systemd-modules-load[335]: Inserted module 'lp'
oct 04 00:31:01 ubuntu-host systemd-modules-load[335]: Inserted module 'ppdev'
oct 04 00:31:01 ubuntu-host systemd-modules-load[335]: Inserted module 'parport_pc'
oct 04 00:31:01 ubuntu-host systemd[1]: Starting Flush Journal to Persistent Storage...

```

Initialisierung und andere vom Betriebssystem ausgegebene Meldungen werden in Dateien im Verzeichnis `/var/log/` gespeichert. Wenn ein kritischer Fehler auftritt und das Betriebssystem nicht in der Lage ist, den Initialisierungsprozess nach dem Laden des Kernels und des `initramfs` fortzusetzen, könnte ein alternatives Bootmedium verwendet werden, um das System zu starten und somit auf das entsprechende Dateisystem zuzugreifen. Dann können die Dateien unter `/var/log/` nach möglichen Gründen für die Unterbrechung des Bootprozesses durchsucht werden. Die Optionen `-D` oder `--directory` des Befehls `journalctl` können benutzt werden, um Logmeldungen in anderen Verzeichnissen als `/var/log/journal/` zu lesen, welches der Standardort für Systemprotokollmeldungen ist. Da die Logmeldungen von `systemd` nicht im Rohformat gespeichert sind, ist der Befehl `journalctl` erforderlich, um sie zu lesen.

Geführte Übungen

1. Wo befindet sich auf einer Maschine, die mit einer BIOS-Firmware ausgestattet ist, das Bootstrap-Binary?

2. Eine UEFI-Firmware unterstützt erweiterte Funktionen, die von externen Programmen, sogenannten EFI-Anwendungen, bereitgestellt werden. Diese Anwendungen haben jedoch ihren eigenen speziellen Ort. Wo auf dem System würden sich die EFI-Anwendungen befinden?

3. Der Bootloader ermöglicht die Übergabe von angepassten Kernelparametern vor dem Laden. Angenommen ein System kann aufgrund einer falschen Partitionszuweisung des Root-Dateisystems nicht booten. Wie würde das korrekte Root-Dateisystem, das sich unter `/dev/sda3` befindet, als Parameter an den Kernel übergeben werden?

4. Der Bootvorgang eines Linux-Rechners endet mit der folgenden Meldung:

```
ALERT! /dev/sda3 does not exist. Dropping to a shell!
```

Was ist die wahrscheinliche Ursache für dieses Problem?

Offene Übungen

1. Der Bootloader zeigt eine Liste von Betriebssystemen zur Auswahl an, wenn mehr als ein Betriebssystem auf dem Rechner installiert ist. Ein neu installiertes Betriebssystem kann jedoch den MBR der Festplatte überschreiben, wodurch die erste Stufe des Bootloaders gelöscht wird und andere Betriebssysteme unzugänglich werden. Warum sollte dies auf einer Maschine, die mit einer UEFI-Firmware ausgestattet ist, nicht passieren?

2. Was ist eine häufige Folge der Installation eines angepassten Kernels, ohne ein entsprechendes initramfs-Image bereitzustellen?

3. Das Initialisierungsprotokoll ist Hunderte von Zeilen lang, daher wird die Ausgabe des Befehls `dmesg` oft an einen Pager weitergeleitet — wie der Befehl `less` — um das Lesen zu erleichtern. Welche `dmesg` Option listet die Ausgabe automatisch, so dass die Notwendigkeit, einen Pager-Befehl explizit zu benutzen, entfällt?

4. Eine Festplatte, die das gesamte Dateisystem einer Offline-Maschine enthielt, wurde entfernt und als sekundäres Laufwerk an eine Arbeitsmaschine angeschlossen. Angenommen, ihr Einhängepunkt ist `/mnt/hd`, wie würde `journalctl` verwendet werden, um den Inhalt der Journaldateien unter `/mnt/hd/var/log/journal/` zu inspizieren?

Zusammenfassung

Diese Lektion erläutert die Bootsequenz in einem Standard-Linux-System. Genaue Kenntnisse darüber, wie der Bootprozess eines Linux-Systems funktioniert, helfen dabei Fehler zu vermeiden, die das System unzugänglich machen können. Die Lektion behandelt die folgenden Themenbereiche:

- Wie sich die BIOS- und UEFI-Bootmethoden unterscheiden.
- Typische Systeminitialisierungsphasen.
- Wiederherstellen von Bootmeldungen.

Die behandelten Befehle und Verfahren lauten:

- Geläufige Kernelparameter.
- Befehle zum Lesen von Bootmeldungen: `dmesg` und `journalctl`.

Lösungen zu den geführten Übungen

1. Wo befindet sich auf einer Maschine, die mit einer BIOS-Firmware ausgestattet ist, das Bootstrap-Binary?

Im MBR des ersten Speichergeräts, welches im BIOS-Konfigurationsprogramm definiert ist.

2. UEFI-Firmware unterstützt erweiterte Funktionen, die von externen Programmen, sogenannten EFI-Anwendungen, bereitgestellt werden. Diese Anwendungen haben jedoch ihren eigenen speziellen Ort. Wo auf dem System würden sich die EFI-Anwendungen befinden?

EFI-Anwendungen werden in der EFI-System-Partition (ESP) gespeichert, die sich an einem beliebigen verfügbaren Speicherblock mit einem kompatiblen Dateisystem (normalerweise ein FAT32-Dateisystem) befindet.

3. Der Bootloader ermöglicht die Übergabe von angepassten Kernelparametern vor dem Laden. Angenommen ein System kann aufgrund einer falschen Partitionszuweisung des Root-Dateisystems nicht booten. Wie würde das korrekte Root-Dateisystem, das sich unter `/dev/sda3` befindet, als Parameter an den Kernel übergeben werden?

Der Parameter `root` mit dem Wert `/dev/sda3` muss verwendet werden, wie in `root=/dev/sda3`.

4. Der Bootvorgang eines Linux-Rechners endet mit der folgenden Meldung:

```
ALERT! /dev/sda3 does not exist. Dropping to a shell!
```

Was ist die wahrscheinliche Ursache für dieses Problem?

Entsprechend der Parameterdefinition des Root-Dateisystems war der Kernel nicht in der Lage, das Gerät `/dev/sda3` zu finden.

Lösungen zu den offenen Übungen

1. Der Bootloader zeigt eine Liste von Betriebssystemen zur Auswahl an, wenn mehr als ein Betriebssystem auf dem Rechner installiert ist. Ein neu installiertes Betriebssystem kann jedoch den MBR der Festplatte überschreiben, wodurch die erste Stufe des Bootloaders gelöscht wird und das andere Betriebssystem unzugänglich wird. Warum sollte dies auf einer Maschine, die mit einer UEFI-Firmware ausgestattet ist, nicht passieren?

UEFI-Maschinen verwenden den MBR der Festplatte nicht, um die erste Stufe des Bootloaders zu speichern.

2. Was ist eine häufige Folge der Installation eines angepassten Kernels, ohne ein entsprechendes initramfs-Image bereitzustellen?

Auf das Root-Dateisystem kann nicht zugegriffen werden, wenn sein Typ als externes Kernelmodul kompiliert wurde.

3. Das Initialisierungsprotokoll ist Hunderte von Zeilen lang, daher wird die Ausgabe des Befehls `dmesg` oft an einen Pager weitergeleitet — wie der Befehl `less` — um das Lesen zu erleichtern. Welche `dmesg` Option listet die Ausgabe automatisch, so dass die Notwendigkeit, einen Pager-Befehl explizit zu benutzen, entfällt?

Die Befehle `dmesg -H` oder `dmesg --human` aktivieren den Pager standardmäßig.

4. Eine Festplatte, die das gesamte Dateisystem einer Offline-Maschine enthielt, wurde entfernt und als sekundäres Laufwerk an eine Arbeitsmaschine angeschlossen. Angenommen, ihr Einhängepunkt ist `/mnt/hd`, wie würde `journalctl` verwendet werden, um den Inhalt der Journaldateien unter `/mnt/hd/var/log/journal/` zu inspizieren?

Mit dem Befehl `journalctl -D /mnt/hd/var/log/journal` oder `journalctl --directory=/mnt/hd/var/log/journal`.